Provas anteriores resolvidas

Programação Orientada a Objetos – Prof. Dr. Sérgio Furgeri

Duas provas do semestre anterior | interpretação, desenvolvimento e código comentado | FATEC Indaiatuba

Este material resolve as duas provas práticas do semestre passado questão por questão. Para cada uma há: o enunciado, a leitura do diagrama UML, a interpretação (o que está sendo pedido e as decisões tomadas), o desenvolvimento passo a passo, o código completo e comentado, o resultado do teste e os pontos em que o professor costuma descontar.

Os códigos são exatamente os dos projetos **ProvaAntiga1** e **ProvaAntiga2** (arquivo Provas_Antigas_Resolvidas.zip). Todos compilam e foram executados com as entradas mostradas, exceto a conexão com o banco de dados da Prova 2, questão 3, que depende do SQL Server instalado (explicado na própria questão).

Mapa das questões

Prova	Questão	Valor	Assunto principal
Prova 1	1 (ex01)	2,0	Herança: Ferrari extends Veiculo, super, @Override, validação
Prova 1	2 (venda)	4,0	Interface FormaPagamento, implements, polimorfismo, JOptionPane
Prova 1	3 (caixa)	4,0	Classes que colaboram, constante, limite de capacidade, menu
Prova 2	1 (ex01)	2,5	Associação 1 para `1..*`: Proprietario com lista de Veiculo
Prova 2	2 (ex02)	2,5	Agregação e import de classe de outro pacote: Frota de Veiculo
Prova 2	3 (ex03)	5,0	Banco de dados com JDBC: inserir e três listagens

O que as duas provas revelam

Formato da prova

- **Prova prática no Eclipse, 3 horas.** A Prova 2 manda, logo no início: crie um projeto com o **seu nome** e coloque toda a prova nele.
- **Uma questão = um pacote.** Os diagramas trazem o nome do pacote na aba do retângulo (`ex01`, `venda`, `caixa`...). Crie o package com esse nome exato.
- **Todas as questões partem de um diagrama UML.** O texto só complementa com as regras. Ler o diagrama corretamente já é metade da nota.
- **Quase sempre há uma classe de teste** (`App` ou "crie uma classe para testar"), com `main`, usando `JOptionPane` ou console.
- **As questões de maior valor (4,0 a 5,0) têm regras**: limite de capacidade, desconto e acréscimo, banco de dados. As de 2,0 a 2,5 são estrutura de classes.

Assuntos cobrados

Assunto	Onde caiu	Está nas listas 1 a 9?
Classe, atributos private, construtor, get/set, validação	Todas as questões	Sim (Listas 7 e 9, Granja)
Herança (extends, super, @Override)	P1 Q1	Não; passado na última aula
Interface e polimorfismo	P1 Q2	Não
Lista de objetos (List / ArrayList)	P2 Q1 e Q2	Parcial (Lista 8 usa List)
Associação, agregação e import entre pacotes	P2 Q1 e Q2	Não
Menu com do-while e switch	P1 Q3	Sim (Listas 6, 7 e 9)
Banco de dados (JDBC)	P2 Q3	Não

> **Atenção:** Interface, associação/agregação e banco de dados não aparecem nas listas 1 a 9. Confirme com a turma o que foi dado nas últimas aulas: o que foi dado tem grande chance de cair, porque as duas provas antigas cobraram matéria das aulas finais.

Como ler um diagrama de classes (UML)

No diagrama	Significado	Em Java
Retângulo com aba e nome (ex01)	Pacote	`package ex01;` (criar o package no Eclipse)
Caixa dividida em 3 partes	Nome / atributos / métodos	`public class Nome { atributos; métodos }`
`-` antes do nome	privado	`private`
`+` antes do nome	público	`public`
`#` antes do nome	protegido (classe e filhas)	`protected`
`nome : String`	atributo e seu tipo	`private String nome;`
`metodo(x : int) : float`	parâmetro e tipo de retorno	`public float metodo(int x)`
`: void`	não devolve nada	`public void metodo()`
Nome em MAIÚSCULAS	constante	`public static final int MAXIMO = 1000;`
Método com o nome da classe	construtor	`public CaixaDeAgua(int qtdeLitros)`

No diagrama	Significado	Em Java
`<<interface>>` em cima do nome	interface	`public interface FormaPagamento`
Linha cheia com triângulo vazado	herança (é um)	`class Filha extends Mae`
Linha tracejada com triângulo vazado	implementa a interface	`class Pix implements FormaPagamento`
Linha tracejada com seta aberta	dependência (usa)	a classe cria ou usa a outra, em geral o App
Linha cheia com seta aberta	associação navegável (tem um)	atributo do tipo da outra classe
Losango vazado	agregação (é formado por)	lista de objetos que existem por conta própria
Losango cheio	composição (parte de)	a parte é criada e morre com o todo
`1`, `1..*`, `0..*`	multiplicidade (quantos)	`1..*` ou `0..*` = lista (`List<Veiculo>`)

Roteiro para qualquer questão

1. Leia o enunciado inteiro e o diagrama antes de abrir o Eclipse. Sublinhe as regras (limites, porcentagens, "não pode").
2. Crie o package com o nome que está na aba do diagrama.
3. Crie primeiro as classes que não dependem de ninguém (a mãe, a interface, a classe modelo), depois as que usam as outras, e por último o `App`.
4. Em cada classe, copie o diagrama de cima para baixo: atributos com o modificador e o tipo certos, depois os métodos com a assinatura exata (nome, parâmetros, retorno).
5. Coloque cada regra do enunciado dentro do método responsável por ela (validação no set ou no método que altera o valor).
6. Escreva o `App` e teste um caso válido e um inválido de cada regra.
7. Comente o código (o professor exige): `/** */` em cima da classe e de cada método, `//` nas etapas.

> **Dica de prova:** Se o diagrama não mostra construtor nem get/set, mas o enunciado exige dar valor a atributos privados ou mostrá-los, acrescente o construtor (ou o get) necessário. Diagramas costumam omitir construtores. Não mude o que está no diagrama: só acrescente o mínimo.

Prova 1 – Questão 1: Veiculo e Ferrari (herança)

Enunciado (2,0)

> 1. Usando a linguagem Java, resolva a questão de acordo com o diagrama.
> a) O método mostrar apresenta em tela os valores do estado do objeto.
> b) A velocidade da Ferrari deve ser um valor entre 0 e 350.
> c) No mesmo pacote, crie uma classe para testar as funcionalidades.

Lendo o diagrama

Elemento	Leitura
Pacote `ex01`	Todas as classes ficam em `package ex01;`
Veiculo: `- placa : String`, `- modelo : String`	Dois atributos privados do tipo String
Veiculo: `+ mostrar() : void`	Método público que não devolve nada
Ferrari: `- velocidade : int`	Atributo privado inteiro, só da Ferrari
Ferrari: `+ definirVelocidade(velocidade : int) : void`	Recebe um int e não devolve nada
Ferrari: `+ mostrar() : void`	A Ferrari também tem mostrar: vai sobrescrever o da mãe
Seta cheia com triângulo vazado de Ferrari para Veiculo	**Herança**: Ferrari é um Veiculo (`extends`)

Interpretação

- Herança significa que a Ferrari ganha tudo o que o Veiculo tem (placa, modelo, mostrar) sem repetir o código, e acrescenta o que é dela (velocidade, definirVelocidade).
- O item b é uma **regra de validação**: dentro de `definirVelocidade`, só aceitar de 0 a 350.
- O item a vale para as duas classes: o `mostrar` da Ferrari precisa mostrar também placa e modelo, que são privados no Veiculo.

> **Decisão tomada:** o diagrama não mostra construtores, mas sem eles não há como dar valor a placa e modelo (são private e não têm set). Foi criado o construtor `Veiculo(placa, modelo)` e o construtor `Ferrari(placa, modelo)` que repassa os dados com `super(...)`.

Desenvolvimento

1. Criar o package `ex01`.
2. Criar `Veiculo` com os dois atributos private, o construtor e o `mostrar()` que imprime placa e modelo.
3. Criar `Ferrari extends Veiculo` com o atributo `velocidade`.
4. No construtor da Ferrari, a primeira linha é `super(placa, modelo);`: ela chama o construtor do Veiculo.
5. Em `definirVelocidade`, um `if (velocidade >= 0 && velocidade <= 350)`; fora disso, mensagem de erro e o valor antigo é mantido.
6. Sobrescrever `mostrar()` com `@Override`: chamar `super.mostrar()` (imprime placa e modelo) e depois imprimir a velocidade.
7. Criar a classe de teste `UsaFerrari` no mesmo pacote (item c), testando uma velocidade válida e duas inválidas.

Código

ex01/Veiculo.java

```
package ex01;

/**
 * Questão 1 - classe Veiculo (superclasse, "classe mãe").
 * <p>
 * Diagrama: atributos privados placa e modelo (String) e o método
 * público mostrar(), que apresenta o estado do objeto.
 * <p>
 * Funcionamento: guarda os dados comuns a qualquer veículo. A classe
 * {@link Ferrari} herda (extends) tudo o que está aqui.
 *
 * @author Warley
 * @version 1.0
 */
public class Veiculo {

    /** Placa do veículo (- no diagrama = private). */
    private String placa;

    /** Modelo do veículo (- no diagrama = private). */
    private String modelo;

    /**
     * Construtor: recebe a placa e o modelo.
     * (Construtores costumam não aparecer no diagrama, mas são
     * necessários para dar valor aos atributos privados.)
     *
     * @param placa  placa do veículo
     * @param modelo modelo do veículo
     */
    public Veiculo(String placa, String modelo) {
        this.placa = placa;
        this.modelo = modelo;
    }

    /**
     * a) Apresenta em tela os valores do estado do objeto
     * (+ no diagrama = public; void = não devolve nada).
     */
    public void mostrar() {
        System.out.println("Placa: " + placa);
        System.out.println("Modelo: " + modelo);
    }
}
```

ex01/Ferrari.java

```
package ex01;

/**
 * Questão 1 - classe Ferrari (subclasse, "classe filha").
 * <p>
 * Diagrama: a seta com triângulo vazado apontando para Veiculo indica
 * HERANÇA: Ferrari é um Veiculo. Em Java isso se escreve com extends.
 * Ferrari ganha placa, modelo e mostrar() de Veiculo e acrescenta o
 * atributo privado velocidade e o método definirVelocidade(int).
 * <p>
 * Regra: a velocidade só aceita valor entre 0 e 350.
 *
 * @author Warley
 * @version 1.0
 */
public class Ferrari extends Veiculo {   // extends = herda de Veiculo

    /** Velocidade atual, de 0 a 350 (- no diagrama = private). */
    private int velocidade;

    /**
     * Construtor: repassa placa e modelo para o construtor de Veiculo.
     *
     * @param placa  placa do carro
     * @param modelo modelo do carro
     */
    public Ferrari(String placa, String modelo) {
        // super(...) chama o construtor da classe mãe; tem de ser a 1ª linha
        super(placa, modelo);
        this.velocidade = 0;   // o carro começa parado
    }

    /**
     * b) Define a velocidade, aceitando só valores entre 0 e 350.
     * Fora da faixa, mostra uma mensagem e mantém o valor anterior.
     *
     * @param velocidade nova velocidade
     */
    public void definirVelocidade(int velocidade) {
        // VALIDAÇÃO da faixa
        if (velocidade >= 0 && velocidade <= 350) {
            this.velocidade = velocidade;
        } else {
            System.out.println("Velocidade inválida! Informe um valor entre 0 e 350.");
        }
    }

    /**
     * Apresenta o estado completo da Ferrari: os dados herdados
     * (placa e modelo) e a velocidade.
     * <p>
     * Sobrescrita (@Override): substitui o mostrar() de Veiculo. Como
     * placa e modelo são private em Veiculo, a Ferrari não enxerga esses
     * atributos; por isso chama super.mostrar() para exibi-los.
     */
    @Override
    public void mostrar() {
        super.mostrar();   // executa o mostrar() da classe mãe (placa e modelo)
        System.out.println("Velocidade: " + velocidade + " km/h");
    }
}
```

ex01/UsaFerrari.java

```
package ex01;

/**
 * Questão 1 - classe de teste (com main).
 * <p>
 * Cria um Veiculo e uma Ferrari, testa a velocidade válida e as
 * inválidas e mostra o estado dos objetos.
 *
 * @author Warley
 * @version 1.0
 */
public class UsaFerrari {

    /**
     * Método principal: ponto de partida do programa.
     *
     * @param args argumentos da linha de comando (não utilizados)
     */
    public static void main(String[] args) {
        // Veiculo comum: só tem placa e modelo
        Veiculo v = new Veiculo("ABC-1234", "Gol");
        System.out.println("=== Veículo ===");
        v.mostrar();

        // Ferrari: herda placa e modelo e ainda tem velocidade
        Ferrari f = new Ferrari("FER-0001", "F8 Tributo");
        f.definirVelocidade(300);   // válida
        System.out.println("\n=== Ferrari ===");
        f.mostrar();

        // TESTE das velocidades inválidas (recusadas, continua 300)
        System.out.println("\n=== Velocidades inválidas ===");
        f.definirVelocidade(400);
        f.definirVelocidade(-10);
        f.mostrar();
    }
}
```

Teste

Saída no Console:

```
=== Veículo ===
Placa: ABC-1234
Modelo: Gol

=== Ferrari ===
Placa: FER-0001
Modelo: F8 Tributo
Velocidade: 300 km/h

=== Velocidades inválidas ===
Velocidade inválida! Informe um valor entre 0 e 350.
Velocidade inválida! Informe um valor entre 0 e 350.
Placa: FER-0001
Modelo: F8 Tributo
Velocidade: 300 km/h
```

Pontos de atenção

Armadilha	Por quê
Escrever `this.placa` dentro da Ferrari	Erro de compilação: placa é private no Veiculo, a filha não enxerga. Use `super.mostrar()` (ou um get).
Esquecer `super(placa, modelo)`	Se o Veiculo só tem construtor com parâmetros, o construtor da filha não compila sem chamar o super.
`super(...)` fora da primeira linha	Erro de compilação: tem que ser a primeira instrução do construtor.
Repetir placa e modelo na Ferrari	Perde o sentido da herança; o professor desconta.
Validar com `||` em vez de `&&`	`v >= 0 || v <= 350` é sempre verdadeiro: aceita qualquer valor.

> **Para decorar:** `class Filha extends Mae` · `super(...)` na 1ª linha do construtor · `@Override` + `super.metodo()` para aproveitar o que a mãe faz.

Prova 1 – Questão 2: Pedido e formas de pagamento (interface)

Enunciado (4,0)

> 2. Utilizando a linguagem Java, codifique o conjunto de elementos apresentados no diagrama ao lado. Considere que a aplicação (App) solicita do usuário (use JOptionPane), o valor do pedido e a forma de pagamento. Considere que o valor do pagamento em cartão deve ser acrescido de 10%, já o pagamento em PIX possui um desconto de 5% em relação ao valor do pedido. Sua aplicação deverá apresentar o valor original do pedido e o valor final (considerando o acréscimo ou desconto).

Lendo o diagrama

Elemento	Leitura
Pacote `venda`	`package venda;` em todas as classes
App: `+ main() : void`	Classe de teste com o main
Seta tracejada de App para Pedido	Dependência: o App cria e usa o Pedido
Pedido: `- valor : float`	Atributo privado do tipo float
Pedido: `+ efetuarPagamento(p : FormaPagamento) : float`	Recebe uma forma de pagamento e devolve o valor final
Losango vazado entre Pedido e FormaPagamento	O pedido trabalha com uma forma de pagamento (aqui ela chega como parâmetro)
`<<interface>> FormaPagamento` com `+ pagar(valor : float) : float`	Interface com um método: recebe o valor e devolve o valor a pagar
Pix e Credito com seta **tracejada** e triângulo vazado	As duas **implementam** a interface (`implements`)

Interpretação

- Uma **interface** é um contrato: diz que toda forma de pagamento tem o método `pagar(float)`, mas não diz como calcular. Cada classe que a implementa escreve o seu cálculo.
- **Credito**: valor + 10%. **Pix**: valor − 5%.
- O ponto central da questão é o **polimorfismo**: `efetuarPagamento` recebe um `FormaPagamento` qualquer e só chama `p.pagar(valor)`. Se o objeto for Pix, roda o cálculo do Pix; se for Credito, o do Credito. O Pedido não precisa de `if` para saber qual é.
- O App pergunta o valor e a forma (JOptionPane), cria o objeto certo, chama o pedido e mostra o valor original e o final.

> **Decisão tomada:** o diagrama não mostra construtor no Pedido; foi criado `Pedido(float valor)`, e também `getValor()` para o App exibir o valor original. Para mostrar dinheiro com 2 casas foi usado `String.format("%.2f", valor)`.

Desenvolvimento

1. Criar o package `venda`.
2. Criar a interface: botão direito no package > New > **Interface**, nome `FormaPagamento`. Dentro, só a assinatura: `float pagar(float valor);` (sem corpo, termina em ponto e vírgula).
3. Criar `Pix implements FormaPagamento` e escrever o `pagar` com o desconto de 5%.
4. Criar `Credito implements FormaPagamento` com o acréscimo de 10%.
5. Criar `Pedido` com o atributo `valor`, o construtor e `efetuarPagamento(FormaPagamento p)` devolvendo `p.pagar(valor)`.
6. Criar o `App`: ler o valor com `Float.parseFloat`, ler a opção, criar `new Pix()` ou `new Credito()` numa variável do tipo `FormaPagamento`, calcular e mostrar.

Código

venda/FormaPagamento.java

```
package venda;

/**
 * Questão 2 - interface FormaPagamento.
 * <p>
 * No diagrama aparece com o estereótipo &lt;&lt;interface&gt;&gt;. Uma interface
 * é um contrato: diz QUAL método toda forma de pagamento precisa ter,
 * mas não diz COMO ele funciona. Cada classe que implementa a interface
 * (Pix e Credito) escreve a sua própria versão do método.
 *
 * @author Warley
 * @version 1.0
 */
public interface FormaPagamento {

    /**
     * Calcula o valor a pagar nesta forma de pagamento.
     * Em interface o método não tem corpo: termina em ponto e vírgula.
     *
     * @param valor valor original do pedido
     * @return valor final, com acréscimo ou desconto
     */
    float pagar(float valor);
}
```

venda/Pix.java

```
package venda;

/**
 * Questão 2 - pagamento em PIX.
 * <p>
 * Seta tracejada com triângulo vazado no diagrama = "implements":
 * Pix cumpre o contrato da interface {@link FormaPagamento}.
 * Regra: PIX tem desconto de 5% sobre o valor do pedido.
 *
 * @author Warley
 * @version 1.0
 */
public class Pix implements FormaPagamento {

    /** Desconto do PIX: 5% (f no final = número do tipo float). */
    private static final float DESCONTO = 0.05f;

    /**
     * Aplica o desconto de 5%.
     *
     * @param valor valor original do pedido
     * @return valor com 5% de desconto
     */
    @Override   // implementa o método que a interface exige
    public float pagar(float valor) {
        return valor - valor * DESCONTO;
    }
}
```

venda/Credito.java

```
package venda;

/**
 * Questão 2 - pagamento em cartão de crédito.
 * <p>
 * Implementa a interface {@link FormaPagamento}.
 * Regra: no cartão o valor tem acréscimo de 10%.
 *
 * @author Warley
 * @version 1.0
 */
public class Credito implements FormaPagamento {

    /** Acréscimo do cartão: 10%. */
    private static final float ACRESCIMO = 0.10f;

    /**
     * Aplica o acréscimo de 10%.
     *
     * @param valor valor original do pedido
     * @return valor com 10% de acréscimo
     */
    @Override
    public float pagar(float valor) {
        return valor + valor * ACRESCIMO;
    }
}
```

venda/Pedido.java

```
package venda;

/**
 * Questão 2 - classe Pedido.
 * <p>
 * Diagrama: atributo privado valor (float) e o método
 * efetuarPagamento(FormaPagamento), que devolve o valor final.
 * A linha com losango vazado entre Pedido e FormaPagamento indica que o
 * pedido USA uma forma de pagamento.
 * <p>
 * O ponto principal da questão: o Pedido não sabe se é Pix ou Credito.
 * Ele só chama p.pagar(valor), e cada forma calcula do seu jeito
 * (isso é polimorfismo).
 *
 * @author Warley
 * @version 1.0
 */
public class Pedido {

    /** Valor original do pedido. */
    private float valor;

    /**
     * Construtor: cria o pedido com o seu valor.
     *
     * @param valor valor original do pedido
     */
    public Pedido(float valor) {
        this.valor = valor;
    }

    /**
     * Devolve o valor original.
     *
     * @return valor do pedido
     */
    public float getValor() {
        return valor;
    }

    /**
     * Efetua o pagamento com a forma recebida.
     *
     * @param p forma de pagamento (um objeto Pix ou Credito)
     * @return valor final, calculado pela própria forma de pagamento
     */
    public float efetuarPagamento(FormaPagamento p) {
        // polimorfismo: chama o pagar() do Pix ou do Credito, conforme o objeto recebido
        return p.pagar(valor);
    }
}
```

venda/App.java

```
package venda;

// Importa a classe JOptionPane, que cria as janelas de entrada e de mensagem
import javax.swing.JOptionPane;

/**
 * Questão 2 - aplicação (App).
 * <p>
 * Solicita o valor do pedido e a forma de pagamento (JOptionPane) e
 * apresenta o valor original e o valor final.
 *
 * @author Warley
 * @version 1.0
 */
public class App {

    /**
     * Método principal: ponto de partida do programa.
     *
     * @param args argumentos da linha de comando (não utilizados)
     */
    public static void main(String[] args) {
        // ENTRADA: valor do pedido (Float.parseFloat converte texto em float)
        float valor = Float.parseFloat(JOptionPane.showInputDialog("Valor do pedido:"));

        // ENTRADA: forma de pagamento
        int opcao = Integer.parseInt(JOptionPane.showInputDialog(
                "Forma de pagamento:\n1 - PIX (5% de desconto)\n2 - Cartão de crédito (10% de
acréscimo)"));

        // A variável é do tipo da INTERFACE: pode guardar um Pix ou um Credito
        FormaPagamento forma;
        if (opcao == 1) {
            forma = new Pix();
        } else if (opcao == 2) {
            forma = new Credito();
        } else {
            JOptionPane.showMessageDialog(null, "Forma de pagamento inválida!");
            return; // encerra o programa
        }

        // Cria o pedido e efetua o pagamento com a forma escolhida
        Pedido pedido = new Pedido(valor);
        float valorFinal = pedido.efetuarPagamento(forma);

        // SAÍDA: String.format("%.2f", ...) mostra com 2 casas decimais
        JOptionPane.showMessageDialog(null,
                "Valor original: R$ " + String.format("%.2f", pedido.getValor())
              + "\nValor final: R$ " + String.format("%.2f", valorFinal));
    }
}
```

Teste

Pedido de 100 no PIX:

```
(janela pergunta: Valor do pedido: -> você digita 100)
(janela pergunta: Forma de pagamento: / 1 - PIX (5% de desconto) / 2 - Cartão de crédito (10% de
acréscimo) -> você digita 1)
(janela mostra) Valor original: R$ 100.00
Valor final: R$ 95.00
```

Pedido de 100 no cartão:

```
(janela pergunta: Valor do pedido: -> você digita 100)
(janela pergunta: Forma de pagamento: / 1 - PIX (5% de desconto) / 2 - Cartão de crédito (10% de
acréscimo) -> você digita 2)
(janela mostra) Valor original: R$ 100.00
Valor final: R$ 110.00
```

Opção inválida:

```
(janela pergunta: Valor do pedido: -> você digita 100)
(janela pergunta: Forma de pagamento: / 1 - PIX (5% de desconto) / 2 - Cartão de crédito (10% de
acréscimo) -> você digita 3)
(janela mostra) Forma de pagamento inválida!
```

Pontos de atenção

Armadilha	Por quê
Esquecer o `f` em `0.05f`	`0.05` é double; colocar double em variável float dá erro de compilação (possible lossy conversion).
Usar `Double.parseDouble` com atributo float	Mesmo problema de tipo. Para float: `Float.parseFloat`.
Colocar corpo no método da interface	Na interface o método termina em `;` (sem chaves).
Esquecer o `public` no `pagar` da classe	O método que implementa a interface tem que ser public, senão não compila.
Fazer `if` dentro do Pedido para saber se é Pix ou Credito	Funciona, mas foge do objetivo (polimorfismo) e perde pontos.
`new FormaPagamento()`	Não se cria objeto de interface; cria-se de uma classe que a implementa.

Para decorar: `interface` = contrato (métodos sem corpo) · `implements` na classe · variável do tipo da interface guarda qualquer classe que a implemente · mesmo comando, comportamento diferente = polimorfismo.

Prova 1 – Questão 3: Caixa d'água e bomba

Enunciado (4,0)

> 3. Utilizando a linguagem Java, codifique o conjunto de elementos apresentados ao lado. A capacidade máxima da caixa deve ser definida como 1000. O método consumir reduz o volume de água da caixa e retorna à quantidade de litros após o consumo. Já o método adicionar permite inserir litros na caixa e retorna a quantidade de litros depois da inclusão. A Bomba deve ser usada para encher a caixa. Crie uma aplicação contendo um menu (use JOptionPane ou modo console), que permita (1) adicionar litros a caixa (2) consumir litros (3) ver a quantidade de litros atual ou (4) encerrar. Não esqueça de controlar o limite de litros para não apresentar vazamento.

Lendo o diagrama

Elemento	Leitura
Pacote `caixa`	`package caixa;`
CaixaDeAgua: `+ CAPACIDADE_MAXIMA : int`	Constante pública (maiúsculas): `public static final int CAPACIDADE_MAXIMA = 1000;`
CaixaDeAgua: `- litrosAtual : int`	Atributo privado com os litros de agora
CaixaDeAgua: `+ adicionar(qtdeLitros : int) : int`	Insere e **devolve** os litros depois
CaixaDeAgua: `+ consumir(qtdeLitros : int) : int`	Retira e **devolve** os litros depois
CaixaDeAgua: `+ CaixaDeAgua(qtdeLitros : int)`	Construtor (mesmo nome da classe, sem tipo de retorno)
Bomba: `+ bombear(caixa : CaixaDeAgua, qtdeLitros : int) : void`	Recebe a caixa e a quantidade; não devolve nada
Linha entre Bomba e CaixaDeAgua	A bomba trabalha com a caixa (recebe como parâmetro)
Setas tracejadas saindo do App	O App usa a Bomba e a CaixaDeAgua

Interpretação

- "Não apresentar vazamento" = a caixa **nunca passa de 1000 litros**. Se pedirem para inserir mais do que cabe, enche até 1000 e avisa o que não coube.
- Pelo mesmo raciocínio, consumir não pode deixar a caixa negativa.
- "A Bomba deve ser usada para encher a caixa": a opção 1 do menu chama `bomba.bombear(caixa, litros)`, e a bomba chama `caixa.adicionar(litros)`. Quem controla o limite é a caixa, porque é ela que tem o atributo.
- O menu é o padrão já treinado: `do { ... switch ... } while (opcao != 4);`.

> **Decisão tomada:** para a opção 3 (ver litros) foi acrescentado `getLitrosAtual()`, que não está no diagrama. Alternativa sem acrescentar nada: chamar `caixa.consumir(0)`, que devolve os litros atuais sem mudar nada. Quantidades negativas são recusadas com mensagem.

Desenvolvimento

1. Criar o package `caixa`.
2. Criar `CaixaDeAgua` com a constante, o atributo e o construtor (que passa pelo `adicionar`, para o limite valer também na criação).
3. Em `adicionar`: se negativo, recusa; se `litrosAtual + qtde` passar de 1000, fica em 1000 e avisa a sobra; senão soma. No fim, `return litrosAtual;`.
4. Em `consumir`: se negativo, recusa; se pedir mais do que tem, zera; senão subtrai. No fim, `return litrosAtual;`.

5. Criar `Bomba` com `bombear(caixa, qtde)` chamando `caixa.adicionar(qtde)`.
6. Criar o `App` com o menu de 4 opções em do-while + switch.

Código

caixa/CaixaDeAgua.java

```
package caixa;

/**
 * Questão 3 - classe CaixaDeAgua.
 * <p>
 * Diagrama: constante pública CAPACIDADE_MAXIMA (1000 litros), atributo
 * privado litrosAtual, construtor CaixaDeAgua(qtdeLitros) e os métodos
 * adicionar e consumir, que devolvem a quantidade de litros depois da
 * operação.
 * <p>
 * Regra: controlar o limite para não haver vazamento (nunca passa de
 * 1000 litros) e nunca ficar com litros negativos.
 *
 * @author Warley
 * @version 1.0
 */
public class CaixaDeAgua {

    /** Capacidade máxima da caixa, em litros (+ no diagrama = public). */
    public static final int CAPACIDADE_MAXIMA = 1000;

    /** Litros que estão na caixa agora (- no diagrama = private). */
    private int litrosAtual;

    /**
     * Construtor: cria a caixa já com uma quantidade inicial de litros.
     * Passa pelo adicionar, então o limite também vale aqui.
     *
     * @param qtdeLitros litros iniciais
     */
    public CaixaDeAgua(int qtdeLitros) {
        this.litrosAtual = 0;
        adicionar(qtdeLitros);
    }

    /**
     * Insere litros na caixa sem passar da capacidade máxima.
     * Se a quantidade não couber, enche até o limite e avisa o que sobrou.
     *
     * @param qtdeLitros litros a inserir
     * @return quantidade de litros depois da inclusão
     */
    public int adicionar(int qtdeLitros) {
        // VALIDAÇÃO: não existe inserir quantidade negativa
        if (qtdeLitros < 0) {
            System.out.println("Quantidade inválida: " + qtdeLitros);
        } else if (litrosAtual + qtdeLitros > CAPACIDADE_MAXIMA) {
            // não cabe tudo: enche até o limite e NÃO vaza
            int sobra = litrosAtual + qtdeLitros - CAPACIDADE_MAXIMA;
            litrosAtual = CAPACIDADE_MAXIMA;
            System.out.println("Caixa cheia! " + sobra + " litro(s) não couberam e não foram inseridos.");
        } else {
            litrosAtual += qtdeLitros;
        }
        return litrosAtual;
    }

    /**
     * Reduz o volume de água da caixa, sem ficar negativo.
     * Se pedir mais do que existe, consome só o que há.
     *
```

```
     * @param qtdeLitros litros a consumir
     * @return quantidade de litros depois do consumo
     */
    public int consumir(int qtdeLitros) {
        if (qtdeLitros < 0) {
            System.out.println("Quantidade inválida: " + qtdeLitros);
        } else if (qtdeLitros > litrosAtual) {
            System.out.println("Só havia " + litrosAtual + " litro(s). A caixa ficou vazia.");
            litrosAtual = 0;
        } else {
            litrosAtual -= qtdeLitros;
        }
        return litrosAtual;
    }

    /**
     * Devolve os litros atuais (método de consulta acrescentado para a
     * opção "ver a quantidade de litros"; não está no diagrama).
     *
     * @return litros na caixa
     */
    public int getLitrosAtual() {
        return litrosAtual;
    }
}
```

caixa/Bomba.java

```
package caixa;

/**
 * Questão 3 - classe Bomba.
 * <p>
 * Diagrama: método bombear(CaixaDeAgua caixa, int qtdeLitros): void.
 * A Bomba é usada para encher a caixa: ela recebe a caixa e chama o
 * método adicionar dela (a caixa é quem controla o limite).
 *
 * @author Warley
 * @version 1.0
 */
public class Bomba {

    /**
     * Bombeia água para dentro da caixa.
     *
     * @param caixa      caixa que vai receber a água
     * @param qtdeLitros litros a bombear
     */
    public void bombear(CaixaDeAgua caixa, int qtdeLitros) {
        // a bomba não mexe direto nos litros: pede para a caixa adicionar
        int litros = caixa.adicionar(qtdeLitros);
        System.out.println("Bomba ligada: caixa com " + litros + " litro(s).");
    }
}
```

caixa/App.java

```
package caixa;

// Importa a classe JOptionPane, que cria as janelas de entrada e de mensagem
import javax.swing.JOptionPane;

/**
 * Questão 3 - aplicação (App) com menu.
 * <p>
 * (1) adicionar litros (usando a Bomba), (2) consumir litros,
 * (3) ver a quantidade de litros atual, (4) encerrar.
 *
 * @author Warley
 * @version 1.0
 */
public class App {

    /**
     * Método principal: ponto de partida do programa.
     *
     * @param args argumentos da linha de comando (não utilizados)
     */
    public static void main(String[] args) {
        // Cria a caixa vazia e a bomba que vai enchê-la
        CaixaDeAgua caixa = new CaixaDeAgua(0);
        Bomba bomba = new Bomba();

        int opcao;
        String menu = "Caixa d'água (capacidade " + CaixaDeAgua.CAPACIDADE_MAXIMA + " litros)\n"
                    + "1 - Adicionar litros\n2 - Consumir litros\n"
                    + "3 - Ver quantidade de litros\n4 - Encerrar";

        // LAÇO do-while: o menu aparece até escolher 4
        do {
            opcao = Integer.parseInt(JOptionPane.showInputDialog(menu));

            switch (opcao) {
                case 1:
                    int entrada = Integer.parseInt(JOptionPane.showInputDialog("Quantos litros
adicionar?"));
                    bomba.bombear(caixa, entrada);   // a bomba enche a caixa
                    JOptionPane.showMessageDialog(null, "Litros na caixa: " + caixa.getLitrosAtual());
                    break;
                case 2:
                    int saida = Integer.parseInt(JOptionPane.showInputDialog("Quantos litros consumir?"));
                    int restante = caixa.consumir(saida);   // consumir devolve os litros após o consumo
                    JOptionPane.showMessageDialog(null, "Litros na caixa: " + restante);
                    break;
                case 3:
                    JOptionPane.showMessageDialog(null, "Litros atuais: " + caixa.getLitrosAtual());
                    break;
                case 4:
                    JOptionPane.showMessageDialog(null, "Programa encerrado.");
                    break;
                default:
                    JOptionPane.showMessageDialog(null, "Opção inválida!");
            }
        } while (opcao != 4);
    }
}
```

Teste

Adiciona 600, tenta adicionar mais 600 (passaria de 1000), vê, consome 300, tenta consumir 900:

```
(janela pergunta: Caixa d'água (capacidade 1000 litros) / 1 - Adicionar litros / 2 - Consumir litros / 3 -
Ver quantidade de litros / 4 - Encerrar -> você digita 1)
(janela pergunta: Quantos litros adicionar? -> você digita 600)
Bomba ligada: caixa com 600 litro(s).
(janela mostra) Litros na caixa: 600
(janela pergunta: Caixa d'água (capacidade 1000 litros) / 1 - Adicionar litros / 2 - Consumir litros / 3 -
Ver quantidade de litros / 4 - Encerrar -> você digita 1)
(janela pergunta: Quantos litros adicionar? -> você digita 600)
Caixa cheia! 200 litro(s) não couberam e não foram inseridos.
Bomba ligada: caixa com 1000 litro(s).
(janela mostra) Litros na caixa: 1000
(janela pergunta: Caixa d'água (capacidade 1000 litros) / 1 - Adicionar litros / 2 - Consumir litros / 3 -
Ver quantidade de litros / 4 - Encerrar -> você digita 3)
(janela mostra) Litros atuais: 1000
(janela pergunta: Caixa d'água (capacidade 1000 litros) / 1 - Adicionar litros / 2 - Consumir litros / 3 -
Ver quantidade de litros / 4 - Encerrar -> você digita 2)
(janela pergunta: Quantos litros consumir? -> você digita 300)
(janela mostra) Litros na caixa: 700
(janela pergunta: Caixa d'água (capacidade 1000 litros) / 1 - Adicionar litros / 2 - Consumir litros / 3 -
Ver quantidade de litros / 4 - Encerrar -> você digita 2)
(janela pergunta: Quantos litros consumir? -> você digita 900)
Só havia 700 litro(s). A caixa ficou vazia.
(janela mostra) Litros na caixa: 0
(janela pergunta: Caixa d'água (capacidade 1000 litros) / 1 - Adicionar litros / 2 - Consumir litros / 3 -
Ver quantidade de litros / 4 - Encerrar -> você digita 4)
(janela mostra) Programa encerrado.
```

Pontos de atenção

Armadilha	Por quê
Esquecer o `return` em adicionar/consumir	O diagrama diz `: int`: o método tem que devolver um inteiro, senão não compila.
Deixar o App mexer direto em litrosAtual	É private; e o controle do limite tem que ficar dentro da caixa.
Não usar a Bomba	O enunciado manda: a bomba enche a caixa. Usar só `caixa.adicionar` no App perde pontos.
Testar só `> 1000` e esquecer do que já tem	O certo é `litrosAtual + qtde > CAPACIDADE_MAXIMA`.
Escrever 1000 solto no código	Use a constante `CAPACIDADE_MAXIMA`; é para isso que ela está no diagrama.

Para decorar: Método com `: int` no diagrama sempre termina com `return` · o limite fica na classe que tem o atributo · menu = do-while + switch + break.

Prova 2 – Instruções gerais

> Antes de iniciar a prova observe as orientações seguintes:
> a) Crie um projeto (no Eclipse) contendo seu nome e adicione toda a prova nesse projeto.
> b) A prova tem duração de 3 horas.

Na prática: **File > New > Java Project**, nome por exemplo `WarleyProva`. Dentro dele, um package para cada questão (`ex01`, `ex02`, `ex03`). As questões 1 e 2 usam a **mesma** classe `Veiculo` (do pacote ex01): a questão 2 importa a classe da questão 1.

Prova 2 – Questão 1: Proprietário e veículos (associação)

Enunciado (2,5)

> 1. Resolva a questão de acordo com o diagrama.
> • O método mostrar deve apresentar em tela os valores atuais do estado do objeto (do tipo Proprietario).
> • A aplicação de teste (App) deve permitir informar os dados do proprietário, seus veículos e apresentar o resultado em tela.

Lendo o diagrama

Elemento	Leitura
Proprietario: `- nome : String`, `+ mostrar() : void`	Atributo privado e método público
Veiculo: `+ placa`, `+ modelo`, `+ Marca` (String)	Atributos **públicos**: acessados direto, sem get/set
Linha "possui" com seta aberta em Veiculo	Associação navegável: o Proprietario conhece os seus veículos
`1` do lado do Proprietario, `1..*` do lado do Veiculo	Um proprietário possui **um ou mais** veículos
`x` perto do Proprietario	O Veiculo não precisa conhecer o dono (navegação só de um lado)
Seta tracejada do App para o Proprietario	O App cria e usa o Proprietario

Interpretação

- "Um ou mais" pede uma **coleção**: o Proprietario guarda uma `List<Veiculo>`. O atributo da lista não aparece desenhado como texto porque a associação já o representa.
- "Estado do objeto" do Proprietario = o nome **e** os veículos dele. O `mostrar` precisa listar todos.
- `1..*` significa no mínimo um veículo: o App não deixa informar zero.

> **Decisão tomada:** o nome `Marca` com M maiúsculo foi mantido exatamente como no diagrama (o padrão Java seria `marca`). Foi criado o construtor `Proprietario(nome, veiculos)` e o `toString()` do Veiculo, para o mostrar ficar simples.

Desenvolvimento

1. Criar o package `ex01`.
2. Criar `Veiculo` com os três atributos public, o construtor e o toString.
3. Criar `Proprietario` com `nome` e `List<Veiculo> veiculos`; importar `java.util.List` e `java.util.ArrayList`.
4. No `mostrar`, imprimir o nome e percorrer a lista com `for (Veiculo v : veiculos)`.
5. No `App`, ler o nome e a quantidade (mínimo 1), ler cada veículo num `for`, adicionar na lista com `add`, criar o proprietário e chamar `mostrar()`.

Código

ex01/Veiculo.java

```
package ex01;

/**
 * Questões 1 e 2 - classe Veiculo.
 * <p>
 * Diagrama: atributos públicos (+) placa, modelo e Marca, todos String.
 * Por serem públicos, podem ser lidos e alterados direto
 * (v.placa = "..."), sem get e set. O nome "Marca" com M maiúsculo foi
 * mantido exatamente como está no diagrama.
 * <p>
 * Também é usada no pacote ex02 (Frota), por meio de import ex01.Veiculo.
 *
 * @author Warley
 * @version 1.0
 */
public class Veiculo {

    /** Placa do veículo. */
    public String placa;
    /** Modelo do veículo. */
    public String modelo;
    /** Marca do veículo (grafia do diagrama). */
    public String Marca;

    /**
     * Construtor com todos os atributos.
     *
     * @param placa  placa
     * @param modelo modelo
     * @param marca  marca
     */
    public Veiculo(String placa, String modelo, String marca) {
        this.placa = placa;
        this.modelo = modelo;
        this.Marca = marca;
    }

    /**
     * Texto com os dados do veículo.
     *
     * @return placa, modelo e marca em uma linha
     */
    @Override
    public String toString() {
        return "Placa: " + placa + " | Modelo: " + modelo + " | Marca: " + Marca;
    }
}
```

ex01/Proprietario.java

```
package ex01;

// ArrayList e List: lista de tamanho variável para guardar os veículos
import java.util.ArrayList;
import java.util.List;

/**
 * Questão 1 - classe Proprietario.
 * <p>
 * Diagrama: atributo privado nome e método mostrar(). A associação
 * "possui", com a seta apontando para Veiculo e multiplicidade 1 para
 * 1..*, significa: um proprietário possui UM OU MAIS veículos. Por isso
 * o Proprietario guarda uma lista de Veiculo.
 *
 * @author Warley
 * @version 1.0
 */
public class Proprietario {

    /** Nome do proprietário. */
    private String nome;

    /** Veículos do proprietário (o lado 1..* da associação). */
    private List<Veiculo> veiculos;

    /**
     * Construtor: recebe o nome e a lista de veículos (pelo menos um, 1..*).
     *
     * @param nome     nome do proprietário
     * @param veiculos veículos que ele possui
     */
    public Proprietario(String nome, List<Veiculo> veiculos) {
        this.nome = nome;
        // copia a lista recebida para dentro do objeto
        this.veiculos = new ArrayList<>(veiculos);
    }

    /**
     * Apresenta em tela os valores atuais do estado do objeto:
     * o nome e os dados de cada veículo.
     */
    public void mostrar() {
        System.out.println("Proprietário: " + nome);
        System.out.println("Quantidade de veículos: " + veiculos.size());
        // for-each: para cada veículo da lista
        for (Veiculo v : veiculos) {
            System.out.println("  " + v);   // chama o toString do Veiculo
        }
    }
}
```

ex01/App.java

```
package ex01;

import java.util.ArrayList;
import java.util.List;
// Importa a classe JOptionPane, que cria as janelas de entrada e de mensagem
import javax.swing.JOptionPane;

/**
 * Questão 1 - aplicação de teste (App).
 * <p>
 * Permite informar os dados do proprietário e dos seus veículos e
 * apresenta o resultado em tela.
 *
 * @author Warley
 * @version 1.0
 */
public class App {

    /**
     * Método principal: ponto de partida do programa.
     *
     * @param args argumentos da linha de comando (não utilizados)
     */
    public static void main(String[] args) {
        // ENTRADA: nome do proprietário
        String nome = JOptionPane.showInputDialog("Nome do proprietário:");

        // ENTRADA: quantidade de veículos (1..* = no mínimo 1)
        int quantidade = Integer.parseInt(JOptionPane.showInputDialog("Quantos veículos ele possui?"));
        while (quantidade < 1) {
            quantidade = Integer.parseInt(JOptionPane.showInputDialog(
                    "O proprietário deve ter pelo menos 1 veículo. Quantos veículos?"));
        }

        // LAÇO: lê os dados de cada veículo e guarda na lista
        List<Veiculo> veiculos = new ArrayList<>();
        for (int i = 1; i <= quantidade; i++) {
            String placa = JOptionPane.showInputDialog("Veículo " + i + " - placa:");
            String modelo = JOptionPane.showInputDialog("Veículo " + i + " - modelo:");
            String marca = JOptionPane.showInputDialog("Veículo " + i + " - marca:");
            veiculos.add(new Veiculo(placa, modelo, marca));
        }

        // Cria o proprietário com seus veículos e mostra o estado
        Proprietario p = new Proprietario(nome, veiculos);
        p.mostrar();
    }
}
```

Teste

Proprietário com 2 veículos:

```
(janela pergunta: Nome do proprietário: -> você digita Warley)
(janela pergunta: Quantos veículos ele possui? -> você digita 2)
(janela pergunta: Veículo 1 - placa: -> você digita ABC-1234)
(janela pergunta: Veículo 1 - modelo: -> você digita Gol)
(janela pergunta: Veículo 1 - marca: -> você digita VW)
(janela pergunta: Veículo 2 - placa: -> você digita XYZ-9876)
(janela pergunta: Veículo 2 - modelo: -> você digita Onix)
(janela pergunta: Veículo 2 - marca: -> você digita Chevrolet)
Proprietário: Warley
Quantidade de veículos: 2
  Placa: ABC-1234 | Modelo: Gol | Marca: VW
  Placa: XYZ-9876 | Modelo: Onix | Marca: Chevrolet
```

Pontos de atenção

Armadilha	Por quê
Guardar só um Veiculo	A multiplicidade é `1..*`: precisa ser lista.
`List<Veiculo> veiculos;` sem `new ArrayList<>()`	A lista fica null e o `add` dá NullPointerException.
Esquecer o import de List/ArrayList	Erro: List cannot be resolved to a type. Ctrl+Shift+O resolve.
Criar get/set para os atributos do Veiculo	Não é erro, mas o diagrama mostra `+`: são públicos, acesso direto.

Prova 2 – Questão 2: Frota (agregação e outro pacote)

Enunciado (2,5)

> 2. Resolva a questão de acordo com o diagrama.
> • O método adicionar permite inserir um veículo a frota.
> • O método mostrar apresenta em tela o conteúdo da frota, incluindo seu nome e dados de cada um de seus veículos.
> • A aplicação de teste (App) permite informar os dados da frota e apresentar esses dados em tela.

Lendo o diagrama

Elemento	Leitura
Dois pacotes: `ex01` (Veiculo) e `ex02` (Frota, App)	A Frota usa uma classe de **outro pacote**: `import ex01.Veiculo;`
Frota: `- empresa : String`	Atributo privado
Frota: `+ adicionar(v : Veiculo) : void`	Recebe um veículo e guarda na frota
Frota: `+ mostrar() : void`	Mostra empresa e veículos
Losango **vazado** do lado da Frota	**Agregação**: a frota é formada por veículos, mas os veículos existem por conta própria
`1` na Frota, `1..*` no Veiculo	Uma frota com um ou mais veículos: lista
Seta tracejada do App para a Frota	O App cria e usa a Frota

Interpretação

- A Frota guarda uma `List<Veiculo>` (agregação = "tem vários").
- Não se cria outra classe Veiculo: a do pacote ex01 é **reaproveitada** com import. É isso que o desenho dos dois pacotes quer mostrar.
- O App lê a empresa e vai adicionando veículos até o usuário dizer que não quer mais.

> **Decisão tomada:** foi criado o construtor `Frota(String empresa)`. Como o import só enxerga o que é public, os atributos e o construtor do Veiculo precisam ser public (e são, pelo diagrama).

Desenvolvimento

1. Criar o package `ex02` no **mesmo projeto** da questão 1.
2. Criar `Frota` com `import ex01.Veiculo;`, o atributo empresa e a lista já criada (`= new ArrayList<>()`).
3. `adicionar(Veiculo v)` faz `veiculos.add(v)`.
4. `mostrar()` imprime a empresa e percorre a lista.
5. Criar o `App` em ex02 com um do-while que lê placa, modelo e marca e pergunta se quer continuar.

Código

ex02/Frota.java

```
package ex02;

// Veiculo está em OUTRO pacote (ex01): precisa ser importado
import ex01.Veiculo;

import java.util.ArrayList;
import java.util.List;

/**
 * Questão 2 - classe Frota.
 * <p>
 * Diagrama: atributo privado empresa e os métodos adicionar(Veiculo) e
 * mostrar(). O losango VAZADO do lado da Frota indica agregação: a frota
 * é formada por veículos (1 frota para 1..* veículos), mas os veículos
 * existem por conta própria. Por isso a Frota guarda uma lista de Veiculo.
 *
 * @author Warley
 * @version 1.0
 */
public class Frota {

    /** Nome da empresa dona da frota. */
    private String empresa;

    /** Veículos da frota. */
    private List<Veiculo> veiculos = new ArrayList<>();

    /**
     * Construtor: cria a frota com o nome da empresa e sem veículos.
     *
     * @param empresa nome da empresa
     */
    public Frota(String empresa) {
        this.empresa = empresa;
    }

    /**
     * Insere um veículo na frota.
     *
     * @param v veículo a inserir
     */
    public void adicionar(Veiculo v) {
        veiculos.add(v);
    }

    /**
     * Apresenta em tela o conteúdo da frota: o nome da empresa e os
     * dados de cada um dos veículos.
     */
    public void mostrar() {
        System.out.println("Frota da empresa: " + empresa);
        System.out.println("Quantidade de veículos: " + veiculos.size());
        for (Veiculo v : veiculos) {
            // atributos públicos de Veiculo: acessados direto
            System.out.println("  Placa: " + v.placa + " | Modelo: " + v.modelo + " | Marca: " + v.Marca);
        }
    }
}
```

ex02/App.java

```
package ex02;

import ex01.Veiculo;
// Importa a classe JOptionPane, que cria as janelas de entrada e de mensagem
import javax.swing.JOptionPane;

/**
 * Questão 2 - aplicação de teste (App).
 * <p>
 * Permite informar os dados da frota e dos veículos (até o usuário
 * responder que não quer mais) e apresenta tudo em tela.
 *
 * @author Warley
 * @version 1.0
 */
public class App {

    /**
     * Método principal: ponto de partida do programa.
     *
     * @param args argumentos da linha de comando (não utilizados)
     */
    public static void main(String[] args) {
        // ENTRADA: empresa e criação da frota
        String empresa = JOptionPane.showInputDialog("Nome da empresa:");
        Frota frota = new Frota(empresa);

        // LAÇO do-while: adiciona veículos até a resposta ser diferente de SIM
        String resposta;
        do {
            String placa = JOptionPane.showInputDialog("Placa:");
            String modelo = JOptionPane.showInputDialog("Modelo:");
            String marca = JOptionPane.showInputDialog("Marca:");
            frota.adicionar(new Veiculo(placa, modelo, marca));

            resposta = JOptionPane.showInputDialog("Adicionar outro veículo? (SIM/NÃO)");
        } while (resposta != null && resposta.equalsIgnoreCase("SIM"));

        // SAÍDA: conteúdo da frota
        frota.mostrar();
    }
}
```

Teste

Frota com 2 veículos:

```
(janela pergunta: Nome da empresa: -> você digita Transportes Silva)
(janela pergunta: Placa: -> você digita AAA-1111)
(janela pergunta: Modelo: -> você digita Actros)
(janela pergunta: Marca: -> você digita Mercedes)
(janela pergunta: Adicionar outro veículo? (SIM/NÃO) -> você digita SIM)
(janela pergunta: Placa: -> você digita BBB-2222)
(janela pergunta: Modelo: -> você digita FH)
(janela pergunta: Marca: -> você digita Volvo)
(janela pergunta: Adicionar outro veículo? (SIM/NÃO) -> você digita NÃO)
Frota da empresa: Transportes Silva
Quantidade de veículos: 2
  Placa: AAA-1111 | Modelo: Actros | Marca: Mercedes
  Placa: BBB-2222 | Modelo: FH | Marca: Volvo
```

Pontos de atenção

Armadilha	Por quê
Copiar a classe Veiculo para dentro de ex02	Funciona, mas ignora o diagrama (Veiculo está em ex01). Use import.
Esquecer o `import ex01.Veiculo;`	Erro: Veiculo cannot be resolved to a type.
Atributos do Veiculo sem public	De outro pacote, só o que é public é visível.
Comparar a resposta com `==`	Texto se compara com `equals` ou `equalsIgnoreCase`.

> **Para decorar:** Agregação (losango vazado) e associação com `1..*` viram `List<Tipo>` + `new ArrayList<>()` + `add` + for-each · classe de outro pacote = `import pacote.Classe;`.

Prova 2 – Questão 3: Banco de dados (JDBC)

Enunciado (5,0)

> 3. Para resolver essa questão:
> a) Elabore um pacote chamado ex03 e acrescente nele todas as classes usadas neste exercício.
> b) Elabore um banco de dados: create database prova
> c) Elabore uma tabela de acordo com o seguinte script: create table blackfriday (id int primary key, produto varchar(50), preco_original money, valor_desconto money)
> d) Elabore as classes e/ou métodos para testar as seguintes funcionalidades:
> I. Inserir um registro e retornar uma mensagem informando o resultado da operação.
> II. Listar todos os registros existentes na tabela blackfriday, apresentando o produto, preco_original, valor_desconto e preco_final (preço original – valor desconto).
> III. Listar todos os registros cujo valor do desconto seja superior ao fornecido pela aplicação.
> IV. Listar todos os registros apresentando o percentual de desconto oferecido para cada produto.

Interpretação

- É metade da nota da prova. Exige três partes: o **banco** (itens b e c, feitos no gerenciador do banco), a **conexão** do Java com o banco e as **classes** que executam os comandos SQL.
- O tipo `money` existe no **SQL Server** (e também no PostgreSQL), não no MySQL. Por isso a solução usa SQL Server. Confirme qual banco existe no laboratório.
- **Preço final** e **percentual** não são colunas: são calculados (no Java ou no SQL). Aqui são calculados na classe `Produto`.
- Item I pede para **retornar uma mensagem**: o método `inserir` devolve uma String com sucesso ou erro.
- Item III: o valor vem da aplicação (o usuário digita) e entra no SQL como parâmetro: `WHERE valor_desconto > ?`.

Conceitos de JDBC que a questão usa

Classe / método	Para que serve
`DriverManager.getConnection(url, usuario, senha)`	Abre a conexão com o banco e devolve um `Connection`
`Connection`	A conexão aberta; dela saem os comandos
`con.prepareStatement(sql)`	Prepara um comando SQL com `?` no lugar dos valores
`ps.setInt(1, x)`, `ps.setString(2, y)`...	Preenche cada `?`, na ordem, começando em 1
`ps.executeUpdate()`	Executa INSERT, UPDATE ou DELETE; devolve quantas linhas mudaram
`ps.executeQuery()`	Executa SELECT; devolve um `ResultSet`
`rs.next()`	Vai para a próxima linha do resultado; false quando acaba (usar em while)
`rs.getString("produto")`, `rs.getDouble(...)`	Lê a coluna da linha atual pelo nome
`SQLException`	Erro do banco; obrigatório tratar com try/catch ou throws
`try ( ... ) { }`	try-with-resources: fecha conexão, comando e resultado sozinho

Organização das classes

Classe	Responsabilidade
`Conexao`	Guarda URL, usuário e senha e abre a conexão
`Produto`	Representa uma linha da tabela; calcula preço final e percentual

Classe	Responsabilidade
`BlackFridayDAO`	Os comandos SQL: inserir (I), listar todos (II e IV), listar desconto maior que (III)
`App`	Menu com JOptionPane que testa as quatro funcionalidades

Desenvolvimento

1. No SQL Server Management Studio, executar o script (itens b e c) e inserir alguns registros de exemplo.
2. Baixar o driver JDBC do SQL Server (arquivo `mssql-jdbc-xx.jar`) e colocar no projeto: botão direito no projeto > **Build Path > Add External Archives...** > escolher o .jar. Sem isso aparece "No suitable driver found".
3. Criar o package `ex03` e a classe `Conexao` com a URL `jdbc:sqlserver://localhost:1433;databaseName=prova;encrypt=false`, usuário e senha.
4. Criar `Produto` com os 4 campos da tabela e os métodos `getPrecoFinal()` e `getPercentualDesconto()`.
5. Criar `BlackFridayDAO`: `inserir` com INSERT e `executeUpdate`; as listagens com SELECT, `executeQuery` e um `while (rs.next())` que monta a lista.
6. Criar o `App` com o menu e um try/catch de `SQLException` para o erro do banco não derrubar o programa.

Script do banco

script_prova.sql

```
-- Questão 3 - itens b) e c). Executar no SQL Server Management Studio.
create database prova;
go
use prova;
go
create table blackfriday (id int primary key, produto varchar(50), preco_original money, valor_desconto
money);
go
-- dados de exemplo para testar as listagens
insert into blackfriday values (1, 'Notebook', 3500.00, 700.00);
insert into blackfriday values (2, 'Fone de ouvido', 200.00, 30.00);
insert into blackfriday values (3, 'Smart TV', 2800.00, 840.00);
```

Código

ex03/Conexao.java

```
package ex03;

// Classes do JDBC (Java Database Connectivity), que já vêm no Java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * Questão 3 - classe responsável por abrir a conexão com o banco "prova".
 * <p>
 * O tipo "money" do script da prova existe no SQL Server, por isso a
 * conexão abaixo é para SQL Server. Para funcionar, o driver JDBC do
 * SQL Server (arquivo mssql-jdbc-xx.jar) precisa estar no Build Path do
 * projeto. Ajuste usuário e senha para os do seu computador/laboratório.
 *
 * @author Warley
 * @version 1.0
 */
public class Conexao {

    /** Endereço do banco: servidor, porta e nome do banco (prova). */
    private static final String URL =
            "jdbc:sqlserver://localhost:1433;databaseName=prova;encrypt=false";
    /** Usuário do banco. */
    private static final String USUARIO = "sa";
    /** Senha do banco. */
    private static final String SENHA = "123456";

    /**
     * Abre e devolve uma conexão com o banco.
     *
     * @return conexão aberta
     * @throws SQLException se não conseguir conectar (servidor desligado,
     *                      senha errada, driver ausente...)
     */
    public static Connection conectar() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, SENHA);
    }
}
```

ex03/Produto.java

```
package ex03;

/**
 * Questão 3 - classe modelo que representa uma linha da tabela blackfriday.
 * <p>
 * Cada atributo corresponde a uma coluna. O preço final e o percentual
 * de desconto não são colunas: são calculados pelos métodos.
 *
 * @author Warley
 * @version 1.0
 */
public class Produto {

    /** Coluna id (chave primária). */
    private int id;
    /** Coluna produto. */
    private String produto;
    /** Coluna preco_original. */
    private double precoOriginal;
    /** Coluna valor_desconto. */
    private double valorDesconto;

    /**
     * Construtor com todas as colunas.
     *
     * @param id            identificação (chave primária)
     * @param produto       nome do produto
     * @param precoOriginal preço sem desconto
     * @param valorDesconto valor do desconto em reais
     */
    public Produto(int id, String produto, double precoOriginal, double valorDesconto) {
        this.id = id;
        this.produto = produto;
        this.precoOriginal = precoOriginal;
        this.valorDesconto = valorDesconto;
    }

    /** @return o id */
    public int getId() { return id; }

    /** @return o nome do produto */
    public String getProduto() { return produto; }

    /** @return o preço original */
    public double getPrecoOriginal() { return precoOriginal; }

    /** @return o valor do desconto */
    public double getValorDesconto() { return valorDesconto; }

    /**
     * Preço final = preço original - valor do desconto.
     *
     * @return preço final
     */
    public double getPrecoFinal() {
        return precoOriginal - valorDesconto;
    }

    /**
     * Percentual de desconto = desconto / preço original * 100.
     *
     * @return percentual de desconto (0 se o preço original for 0)
     */
```

```
    public double getPercentualDesconto() {
        if (precoOriginal == 0) {
            return 0;   // evita divisão por zero
        }
        return valorDesconto / precoOriginal * 100;
    }
}
```

ex03/BlackFridayDAO.java

```
package ex03;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

/**
 * Questão 3 - classe de acesso aos dados (DAO = Data Access Object).
 * <p>
 * Concentra todos os comandos SQL da tabela blackfriday: inserir e as
 * três listagens pedidas. Usa PreparedStatement com "?" no lugar dos
 * valores, que é o jeito seguro de passar dados para o SQL.
 * <p>
 * O try (...) { } com parênteses (try-with-resources) fecha a conexão,
 * o comando e o resultado automaticamente no final.
 *
 * @author Warley
 * @version 1.0
 */
public class BlackFridayDAO {

    /**
     * I. Insere um registro e devolve uma mensagem com o resultado.
     *
     * @param p produto a inserir
     * @return mensagem de sucesso ou de erro
     */
    public String inserir(Produto p) {
        String sql = "INSERT INTO blackfriday (id, produto, preco_original, valor_desconto) VALUES
(?, ?, ?, ?)";
        try (Connection con = Conexao.conectar();
             PreparedStatement ps = con.prepareStatement(sql)) {
            // cada ? recebe um valor, na ordem (1, 2, 3, 4)
            ps.setInt(1, p.getId());
            ps.setString(2, p.getProduto());
            ps.setDouble(3, p.getPrecoOriginal());
            ps.setDouble(4, p.getValorDesconto());
            // executeUpdate executa INSERT/UPDATE/DELETE e devolve quantas linhas mudaram
            int linhas = ps.executeUpdate();
            return linhas + " registro inserido com sucesso!";
        } catch (SQLException e) {
            // ex.: id repetido (chave primária) ou banco fora do ar
            return "Falha ao inserir: " + e.getMessage();
        }
    }

    /**
     * II e IV. Lista todos os registros da tabela.
     * O preço final e o percentual são calculados pela classe Produto.
     *
     * @return lista com todos os produtos
     * @throws SQLException se houver erro no banco
     */
    public List<Produto> listarTodos() throws SQLException {
        String sql = "SELECT id, produto, preco_original, valor_desconto FROM blackfriday ORDER BY id";
        try (Connection con = Conexao.conectar();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {   // executeQuery = SELECT
            return montarLista(rs);
        }
```

```
    }

    /**
     * III. Lista os registros cujo valor do desconto é maior que o informado.
     *
     * @param valorMinimo valor de desconto fornecido pela aplicação
     * @return produtos com desconto maior que esse valor
     * @throws SQLException se houver erro no banco
     */
    public List<Produto> listarDescontoMaiorQue(double valorMinimo) throws SQLException {
        String sql = "SELECT id, produto, preco_original, valor_desconto FROM blackfriday "
                   + "WHERE valor_desconto > ? ORDER BY id";
        try (Connection con = Conexao.conectar();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setDouble(1, valorMinimo);
            try (ResultSet rs = ps.executeQuery()) {
                return montarLista(rs);
            }
        }
    }

    /**
     * Transforma cada linha do ResultSet em um objeto Produto.
     *
     * @param rs resultado da consulta
     * @return lista de produtos
     * @throws SQLException se houver erro ao ler as colunas
     */
    private List<Produto> montarLista(ResultSet rs) throws SQLException {
        List<Produto> lista = new ArrayList<>();
        // rs.next() avança para a próxima linha; devolve false quando acabam
        while (rs.next()) {
            lista.add(new Produto(
                    rs.getInt("id"),
                    rs.getString("produto"),
                    rs.getDouble("preco_original"),
                    rs.getDouble("valor_desconto")));
        }
        return lista;
    }
}
```

ex03/App.java

```
package ex03;

import java.sql.SQLException;
import java.util.List;
// Importa a classe JOptionPane, que cria as janelas de entrada e de mensagem
import javax.swing.JOptionPane;

/**
 * Questão 3 - aplicação de teste com menu para as quatro funcionalidades.
 *
 * @author Warley
 * @version 1.0
 */
public class App {

    /**
     * Método principal: ponto de partida do programa.
     *
     * @param args argumentos da linha de comando (não utilizados)
     */
    public static void main(String[] args) {
        BlackFridayDAO dao = new BlackFridayDAO();
        String menu = "Black Friday\n"
                    + "1 - Inserir registro\n"
                    + "2 - Listar todos (com preço final)\n"
                    + "3 - Listar com desconto maior que um valor\n"
                    + "4 - Listar com percentual de desconto\n"
                    + "0 - Sair";
        int opcao;

        do {
            opcao = Integer.parseInt(JOptionPane.showInputDialog(menu));
            try {
                switch (opcao) {
                    case 1: {
                        // I. lê os dados e insere
                        int id = Integer.parseInt(JOptionPane.showInputDialog("Id:"));
                        String nome = JOptionPane.showInputDialog("Produto:");
                        double preco = Double.parseDouble(JOptionPane.showInputDialog("Preço original:"));
                        double desconto = Double.parseDouble(JOptionPane.showInputDialog("Valor do
desconto:"));
                        String msg = dao.inserir(new Produto(id, nome, preco, desconto));
                        JOptionPane.showMessageDialog(null, msg);
                        break;
                    }
                    case 2: {
                        // II. produto, preço original, desconto e preço final
                        String texto = "";
                        for (Produto p : dao.listarTodos()) {
                            texto += p.getProduto()
                                   + " | Original: R$ " + String.format("%.2f", p.getPrecoOriginal())
                                   + " | Desconto: R$ " + String.format("%.2f", p.getValorDesconto())
                                   + " | Final: R$ " + String.format("%.2f", p.getPrecoFinal()) + "\n";
                        }
                        mostrar(texto);
                        break;
                    }
                    case 3: {
                        // III. desconto maior que o valor informado
                        double valor = Double.parseDouble(JOptionPane.showInputDialog("Mostrar descontos
maiores que R$:"));
                        String texto = "";
                        for (Produto p : dao.listarDescontoMaiorQue(valor)) {
```

```
                            texto += p.getProduto() + " | Desconto: R$ " + String.format("%.2f",
p.getValorDesconto()) + "\n";
                        }
                        mostrar(texto);
                        break;
                    }
                    case 4: {
                        // IV. percentual de desconto de cada produto
                        String texto = "";
                        for (Produto p : dao.listarTodos()) {
                            texto += p.getProduto() + " | Desconto: "
                                    + String.format("%.1f", p.getPercentualDesconto()) + "%\n";
                        }
                        mostrar(texto);
                        break;
                    }
                    case 0:
                        break;
                    default:
                        JOptionPane.showMessageDialog(null, "Opção inválida!");
                }
            } catch (SQLException e) {
                // erro de banco não derruba o programa: mostra e volta ao menu
                JOptionPane.showMessageDialog(null, "Erro no banco de dados: " + e.getMessage());
            }
        } while (opcao != 0);
    }

    /**
     * Mostra o texto de uma listagem, ou um aviso se ela vier vazia.
     *
     * @param texto linhas da listagem
     */
    private static void mostrar(String texto) {
        JOptionPane.showMessageDialog(null, texto.isEmpty() ? "Nenhum registro encontrado." : texto);
    }
}
```

Teste

O código compila. A execução completa depende de um SQL Server instalado, que não estava disponível no ambiente de teste. O que foi verificado: os comandos SQL rodaram em outro banco com os mesmos dados de exemplo (resultados abaixo), e sem o driver o programa mostra a mensagem de erro e volta ao menu em vez de travar.

Produto	Original	Desconto	Final (II)	Percentual (IV)
Notebook	R$ 3500,00	R$ 700,00	R$ 2800,00	20,0%
Fone de ouvido	R$ 200,00	R$ 30,00	R$ 170,00	15,0%
Smart TV	R$ 2800,00	R$ 840,00	R$ 1960,00	30,0%

Item III com valor 100: aparecem **Notebook** e **Smart TV** (descontos de 700 e 840). Item I com id repetido: o banco recusa (chave primária) e o método devolve "Falha ao inserir: ...".

Execução sem o driver no Build Path:

```
(janela pergunta: Black Friday / 1 - Inserir registro / 2 - Listar todos (com preço final) / 3 - Listar
com desconto maior que um valor / 4 - Listar com percentual de desconto / 0 - Sair -> você digita 2)
(janela mostra) Erro no banco de dados: No suitable driver found for
jdbc:sqlserver://localhost:1433;databaseName=prova;encrypt=false
(janela pergunta: Black Friday / 1 - Inserir registro / 2 - Listar todos (com preço final) / 3 - Listar
com desconto maior que um valor / 4 - Listar com percentual de desconto / 0 - Sair -> você digita 0)
```

Se o banco do laboratório for outro

Banco	O que muda em Conexao.java
SQL Server	`jdbc:sqlserver://localhost:1433;databaseName=prova;encrypt=false` (driver mssql-jdbc)
PostgreSQL	`jdbc:postgresql://localhost:5432/prova` (driver postgresql-xx.jar). No item III troque `?` por `?::money` ou use a coluna como numeric.
MySQL	`jdbc:mysql://localhost:3306/prova` (driver mysql-connector-j). MySQL não tem `money`: use `decimal(10,2)` no script.

Pontos de atenção

Armadilha	Por quê
No suitable driver found	O .jar do driver não está no Build Path do projeto.
Login failed / connection refused	Usuário, senha ou porta errados, ou o serviço do banco desligado.
`executeQuery` em INSERT	INSERT usa `executeUpdate`; SELECT usa `executeQuery`.
Montar o SQL somando texto (`"... > " + valor`)	Funciona, mas é inseguro (SQL injection); o professor espera `?` com PreparedStatement.
Esquecer `rs.next()`	O ResultSet começa antes da primeira linha; sem next não há dados.
Não tratar SQLException	Não compila: é exceção verificada (try/catch ou throws).
Esquecer o `create database` antes do `use prova`	A tabela é criada no banco errado (master).

Para decorar: Conectar (DriverManager) · preparar (prepareStatement com ?) · preencher (setX) · executar (executeUpdate para INSERT, executeQuery para SELECT) · ler (while rs.next() + getX) · fechar (try-with-resources).

Checklist do dia da prova

- Projeto com o seu nome; um package por questão com o nome do diagrama.
- Cada atributo com o modificador do diagrama (`-` private, `+` public) e o tipo exato (float é float, int é int).
- Cada método com o nome, os parâmetros e o retorno do diagrama; método com tipo diferente de void termina em `return`.
- Setas: cheia com triângulo = `extends`; tracejada com triângulo = `implements`; `1..*` ou losango = lista.
- Toda regra do enunciado dentro de um método, com `if` e mensagem.
- Classe de teste com `main` em toda questão que pedir (ou que tenha `App` no diagrama).
- Testar um caso válido e um inválido de cada regra antes de entregar.
- Comentários: `/** */` na classe e nos métodos, `//` nas etapas.
- Salvar tudo (Ctrl+Shift+S) e conferir que não há nenhum X vermelho no Package Explorer.